Biolyt Retrieval Platform

Technical Architecture Overview — Retrieval-Augmented Generation Layer

Internal Engineering Documentation · Revision 1.0

93,435	3,240,756	1,024-d	~300 ms
SOURCE DOCUMENTS	INDEXED CHUNKS	DENSE + SPARSE VECTORS	MEDIAN QUERY LATENCY

1. Introduction

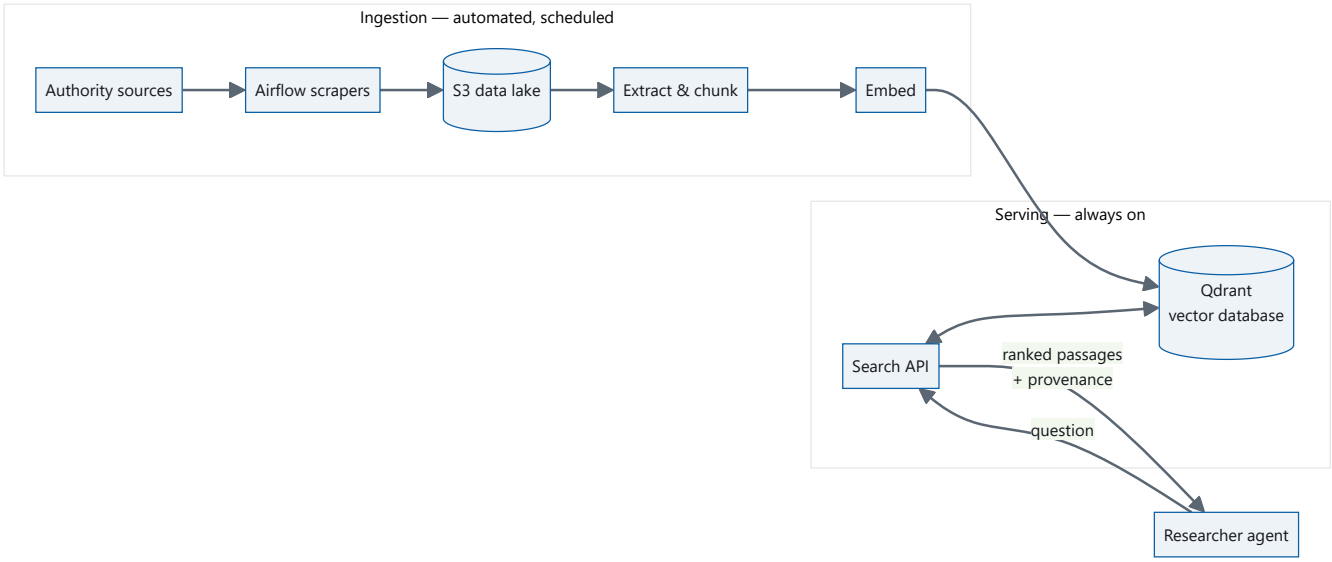
The Biolyt Retrieval Platform is the semantic knowledge layer of the main platform. Its sole consumer is the **researcher agent**, which answers regulatory and clinical questions on behalf of end users. The platform converts a natural-language question into a ranked, fully attributed set of source passages that the agent uses as grounding context.

Business problem

Pharmaceutical regulatory intelligence is distributed across tens of thousands of authority-published PDFs — Summaries of Product Characteristics, Patient Information Leaflets, Public Assessment Reports and agency assessment dossiers. The information is authoritative but unsearchable in any practical sense: keyword search over PDFs cannot connect "*is this safe while breastfeeding*" to a section headed "*Fertility, pregnancy and lactation*", and an analyst cannot read 93,000 documents.

The platform closes that gap. Every answer the researcher agent produces is traceable to an exact document, page and regulatory section — a requirement in this domain, where an unsourced claim carries no weight.

Architecture at a glance



Two independent planes. The **ingestion plane** runs on a schedule and keeps the index aligned with the authorities' published state. The **serving plane** runs continuously and answers queries in milliseconds. Neither blocks the other: indexing new material never interrupts retrieval.

2. Document Corpus

The knowledge base is assembled from eight curated domains, each mapped to a dedicated prefix in the S3 data lake and populated by its own scraper.

Domain	Content
Regulatory Approvals	SPCs, PILs and Public Assessment Reports from the EMA and MHRA
Clinical Trials & Pipeline Intelligence	Registry records, protocols and pipeline disclosures
Safety & Pharmacovigilance	Signal detection, adverse-event and post-marketing surveillance material
Drug Substance Reference	Chemical, structural and formulation reference data
Targets, Genomics & Biomarkers	Target validation, variant and biomarker evidence
Literature Evidence	Indexed scientific literature and bibliographic metadata
MENA & GCC Regulatory Market	Regional authority filings and market authorisations
Ontologies & Standards	Controlled vocabularies and terminology standards

Document acquisition is fully automated. Each source is defined by a declarative manifest and executed as a task group inside an Apache Airflow DAG. Scrapers publish through an **atomic commit protocol**: output is staged under a run-scoped prefix, verified byte-for-byte and by SHA-256 checksum, promoted

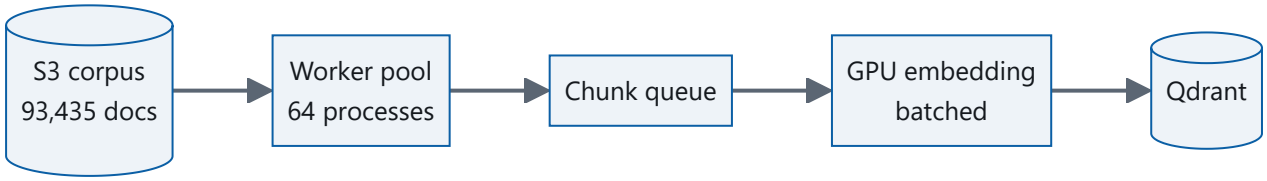
to the live view only on success, and finally advertised by flipping a `_LATEST` pointer. A failed or partial run is therefore invisible to downstream consumers — the index never observes a half-written source.

Completed scraper runs emit an Airflow **Dataset** signal. The indexing DAG subscribes to those signals, so new documents flow into the vector store without a human trigger and without a fixed coupling between scraping and indexing schedules.

3. Initial Corpus Processing

Bringing an empty index to full coverage is a fundamentally different workload from keeping it current: **93,435 documents** had to be downloaded, parsed, segmented and embedded once. This was executed as a discrete, one-time initialisation.

The bottleneck is asymmetric. Text extraction and segmentation are CPU-bound and highly parallel; embedding generation is GPU-bound and benefits from large batches. The initialisation pipeline separates the two so neither waits on the other:



Sixty-four worker processes handled download, PDF text extraction and segmentation in parallel, streaming chunks to a single GPU process that embedded them in large batches and wrote to the vector database asynchronously.

Parameter	Value
Compute	RunPod GPU instance — NVIDIA RTX 4090, 120 vCPU
Parallelism	64 extraction workers, 1 GPU embedding process
Documents processed	93,435
Chunks generated	3,240,756
Wall-clock duration	3 h 47 min
Model	BAAI/bge-m3, FP16

GPU capacity is treated as a **burst resource**. The instance is provisioned for the duration of the job and released on completion, so the platform carries no standing GPU cost. Steady-state operation — incremental indexing and all query serving — runs on standard compute.

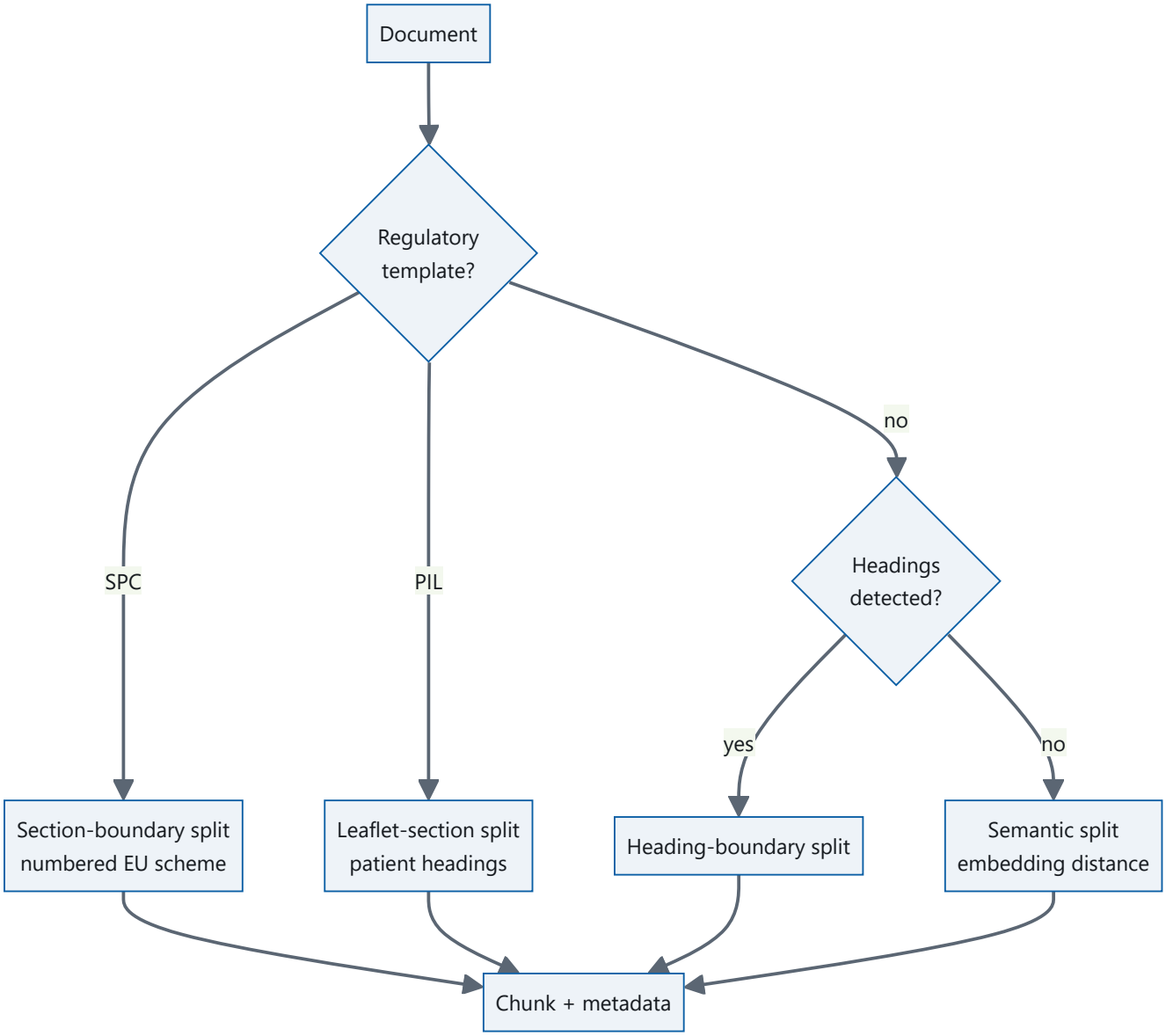
> **Idempotent by construction.** Every chunk's primary key is derived deterministically from its document identity and position, so re-processing a document overwrites rather than duplicates. Initialisation is therefore safely resumable: an interrupted run continues from where it stopped instead of restarting.

4. Chunking Strategy

Segmentation determines the ceiling on retrieval quality. A chunk that straddles two regulatory sections dilutes both; a chunk that splits a contraindication mid-sentence retrieves as neither.

Fixed-size chunking was rejected because it discards the single most valuable property of this corpus: **regulatory documents are highly structured**. An EU Summary of Product Characteristics follows a mandated numbering scheme — 4.3 Contraindications, 4.5 Interactions, 4.8 Undesirable effects — that is consistent across every product and every national authority. Cutting at a fixed token count destroys that structure; cutting *along* it preserves a natural, semantically coherent unit and yields precise metadata for free.

The platform therefore applies a **structure-aware cascade**, selecting the most specific strategy each document supports and degrading gracefully when structure is absent:



Semantic splitting

Where a document carries no usable structure — narrative assessment reports, scanned filings, unformatted submissions — the platform falls back to **semantic segmentation**. Consecutive sentences are embedded and the cosine distance between neighbours is measured; boundaries are placed where that distance spikes, marking a genuine shift in subject matter.

The cut-off is expressed as a **percentile of each document's own distance distribution** rather than an absolute constant, because distance scales differ between documents. A fixed value over-cuts some documents into fragments and under-cuts others into monoliths; a percentile adapts to each document automatically. The operating percentile was selected empirically by sweeping candidate values across representative documents and measuring the resulting size distributions.

Overlap and context preservation

Chunks carry a configurable token overlap at their boundaries. A statement whose meaning depends on the sentence preceding it — a dose qualified by the patient population named just above it — remains interpretable in isolation, because the qualifying context travels with the chunk. Overlap is applied where boundaries are inferred rather than mandated; a chunk cut on a regulatory section boundary is already a complete semantic unit and needs no bleed from its neighbour.

Outcome

Strategy applied	Chunks	Share
SPC section boundaries	1,471,939	45.4%
PIL leaflet sections	778,565	24.0%
Semantic boundaries	548,509	16.9%
Heading boundaries	441,743	13.6%
Total	**3,240,756**	**100%**

Every chunk in the corpus was produced by a deliberate structural or semantic decision. The strategy that produced each chunk is recorded in its payload, making segmentation behaviour measurable across the whole index rather than assumed.

5. Embedding Pipeline

The platform standardises on **BAAI/bge-m3** for all embedding generation. Queries and documents pass through the same model, guaranteeing that both occupy the same vector space.

Property	Value
Model	BAAI/bge-m3
Dense dimension	1,024
Sparse representation	Learned lexical weights, emitted in the same pass
Context window	8,192 tokens
Distance metric	Cosine
Precision	FP16 on GPU

Why bge-m3

- **Dual representation from a single forward pass.** bge-m3 emits a dense semantic vector *and* a sparse lexical vector together. Most embedding models provide only the dense half, requiring a separate lexical index to be built, stored and kept in sync. Here one model and one pass produce both.
- **Lexical precision where it matters.** Dense vectors generalise well but blur rare tokens — molecule names, licence numbers, ATC codes. The sparse component matches those exactly, which is essential in a corpus where `PL 20092/0008` identifies a specific authorisation.
- **Multilingual capability.** The model is trained across 100+ languages in a shared space, allowing regional filings to be indexed alongside English-language material without a separate pipeline.
- **Long context.** An 8,192-token window accommodates full regulatory sections without forced truncation.

6. Vector Database

Qdrant serves as the vector database, chosen for native support of the two properties this workload depends on: hybrid dense-plus-sparse vectors in a single collection, and rich payload filtering applied *before* the vector search rather than after it.

Storage and indexing

Mechanism	Purpose
HNSW graph index	Approximate nearest-neighbour search at millions-of-vectors scale
INT8 scalar quantization	Compresses the searchable representation ~4×, pinned in RAM for fast traversal
Full-precision on disk	Originals retained on disk, keeping the memory footprint independent of corpus size
On-disk payloads	Chunk text stored on disk and fetched only for returned results
Named vectors	Dense and sparse vectors co-located on every point

The result is a 3.2-million-vector index that serves sub-second queries from commodity hardware, with memory consumption driven by the quantized representation rather than the raw corpus.

Payload metadata and filtering

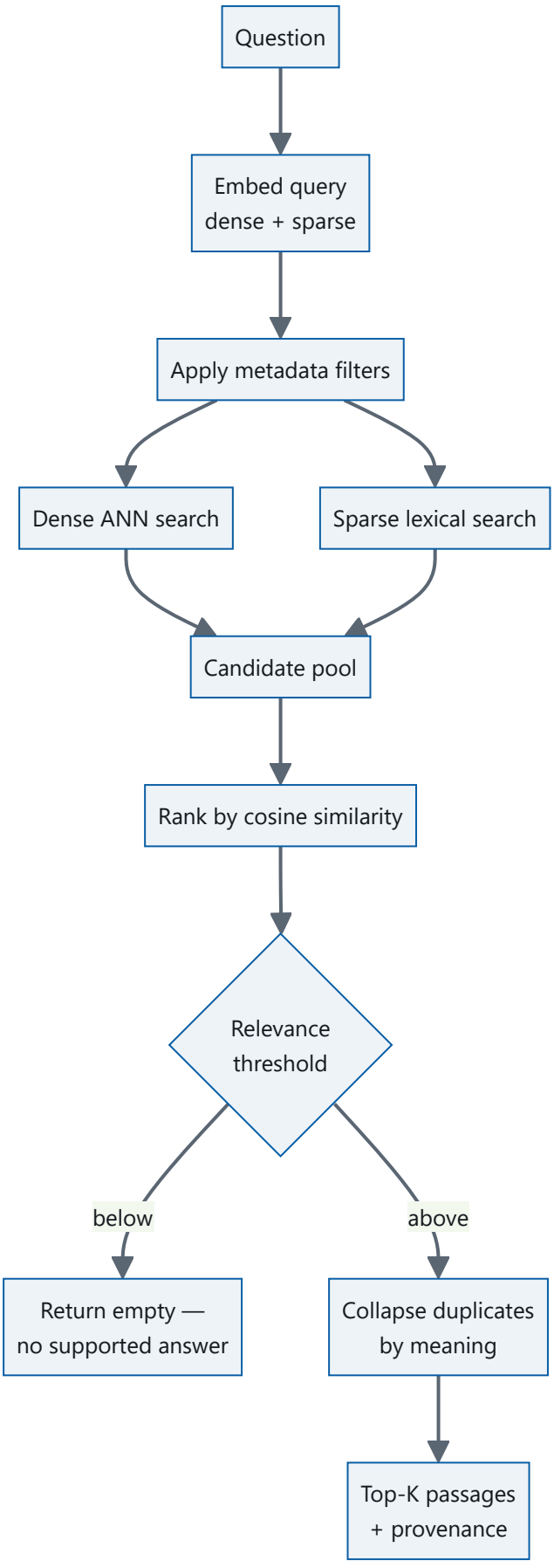
Every chunk carries structured metadata, each field independently indexed so filters narrow the candidate space before any vector comparison occurs:

Field	Role
`section`, `section_code`	Regulatory section and its EU number (e.g. `4.8`) — scope a query to contraindications, posology or adverse effects
`doc_type`	SPC, PIL or assessment report
`source`, `s3_key`, `page`	Full provenance for citation
`chunk_path`	Segmentation strategy that produced the chunk
`etag`, `offset`	Change detection and incremental sync
`molecule_id`, `language`	Entity scoping and language selection

This is what elevates the platform above generic semantic search. A question such as "*what are the contraindications in hepatic impairment*" is not answered by similarity alone — it is scoped to section 4.3 across the entire corpus, then ranked by meaning within that scope.

7. Retrieval Pipeline

Retrieval is a staged narrowing: from 3.2 million chunks to a handful of passages, with each stage cheaper per candidate than the one before it.


Question
Embed query
dense + sparse
Apply metadata filters
Dense ANN search
Sparse lexical search
Candidate pool
Rank by cosine similarity
Relevance
threshold
below
above
Return empty —
no supported answer
Collapse duplicates
by meaning
Top-K passages
+ provenance

Stage	Function
Query embedding	The question is encoded by the same model used at index time, producing dense and sparse vectors in one pass.
Filtering	Any supplied metadata constraint is applied first, reducing the search space before vector comparison.
Hybrid candidate retrieval	Dense and sparse searches run in parallel. Dense contributes conceptual matches; sparse contributes exact-term matches the dense space would blur.
Similarity ranking	All candidates, regardless of which search surfaced them, are scored by cosine similarity against the query vector — one comparable measure across the whole pool.
Relevance gating	A calibrated similarity floor determines whether the corpus supports an answer at all. Below it, the API returns nothing rather than weak matches.
Duplicate collapse	Passages conveying the same fact are merged, with the additional sources retained as corroborating citations.
Context assembly	The surviving passages are returned with full provenance, ready for grounding.

> **Corroboration as signal.** Regulatory text is intentionally duplicated — generic medicines are legally required to reproduce the originator's wording, so a single contraindication may appear across dozens of authorisations. Rather than discarding those copies, the platform collapses them and returns the additional sources alongside. "*Consistent across every UK authorisation*" is a stronger evidentiary statement than a single citation.

8. Similarity Search

Ranking is performed by **cosine similarity** — the cosine of the angle between the query vector and a chunk vector, independent of their magnitudes.

That independence is the reason it suits dense retrieval. Embedding magnitude correlates with incidental properties such as passage length; direction encodes meaning. Cosine compares direction alone, so a concise contraindication and a lengthy assessment paragraph expressing the same concept score comparably. The collection is configured with cosine distance natively, and vectors are normalised so the metric is applied consistently at index and query time.

Because cosine produces a bounded, comparable score for every candidate, it also underpins two capabilities beyond ordering: a calibrated **relevance floor** that distinguishes a supported question from an unsupported one, and **duplicate detection**, where near-identical passages are identified by the similarity of their vectors rather than their literal text.

9. Reranking

The platform includes a **cross-encoder reranking** stage, available as a precision enhancement over the retrieved candidate set.

Property	Value
Model	BAAI/bge-reranker-v2-m3
Architecture	Cross-encoder
Input	Query and candidate passage jointly
Placement	After candidate retrieval, before context assembly
Execution	GPU-accelerated; enabled per deployment

Why reranking raises precision

The retrieval model is a *bi-encoder*: query and passage are encoded **separately** and compared afterwards. Each is compressed into 1,024 numbers before the two ever meet, so the comparison measures topical proximity rather than whether the passage answers the question.

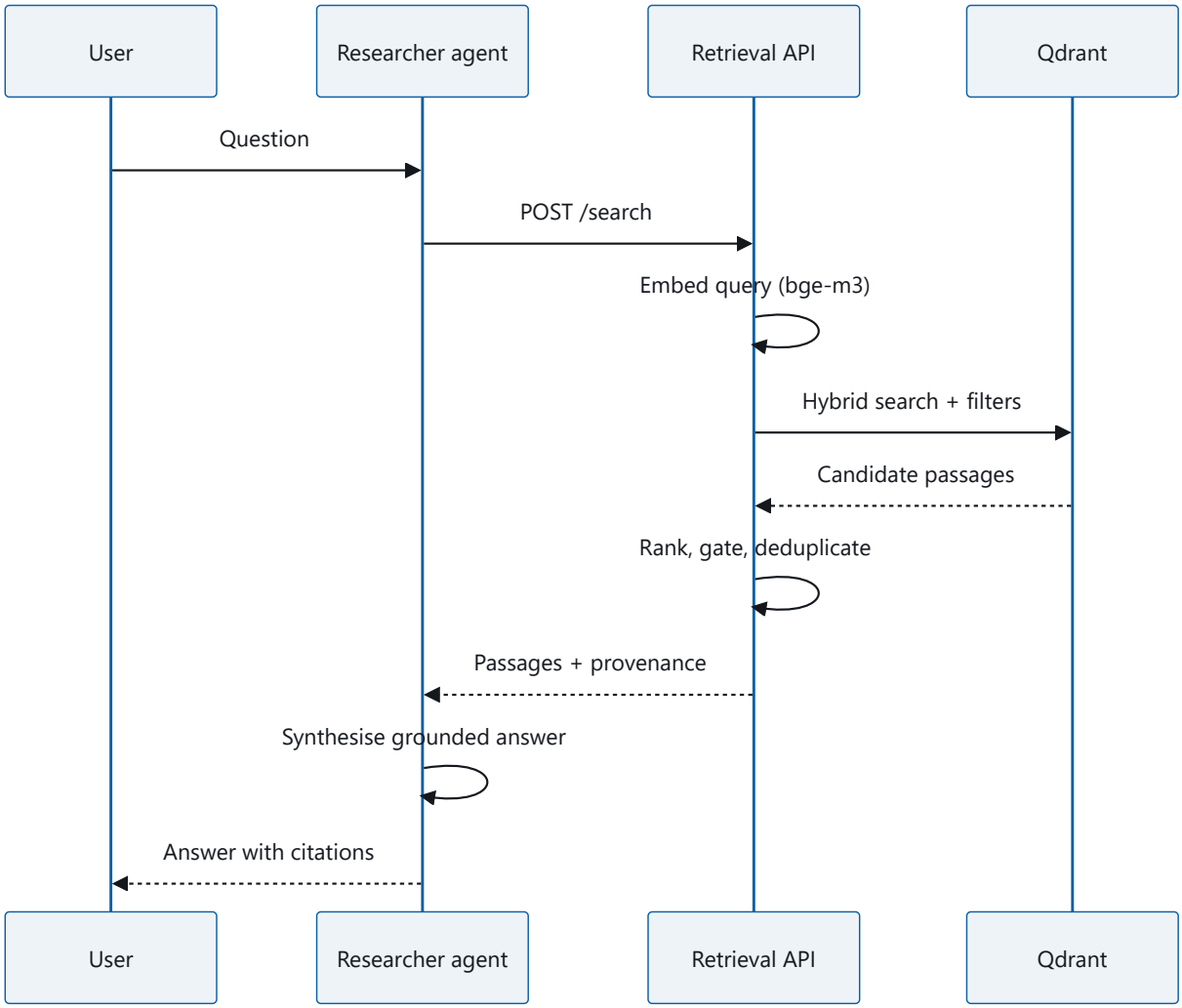
A cross-encoder removes that separation. Query and passage are processed **together**, with full attention between every query token and every passage token. It can therefore resolve distinctions a bi-encoder structurally cannot:

- **Negation.** "*Contraindicated in pregnancy*" and "*may be used in pregnancy*" are topically near-identical and semantically opposite.
- **Entity binding.** A question naming one molecule should not be satisfied by an otherwise identical statement about a different one.
- **Answer-bearing vs. merely related.** Risk factors for a condition are related to, but are not, its clinical signs.

This precision is purchased with computation — the cross-encoder performs one forward pass per candidate rather than one per query — which is why it is positioned as the final, narrowest stage of the funnel, operating on a pre-filtered candidate set rather than the corpus.

10. Inference Pipeline

End to end, from question to grounded answer:



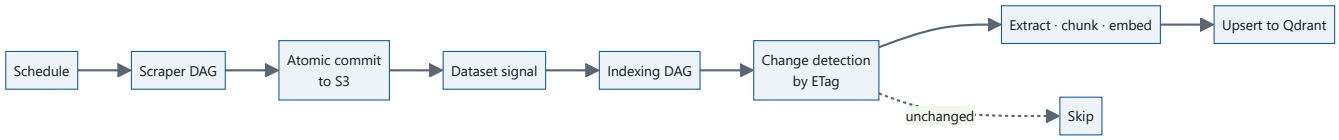
Stage	Component	Responsibility
Query understanding	Researcher agent	Interprets intent; may scope the request to a regulatory section
Query encoding	bge-m3	Dense + sparse query representation
Candidate retrieval	Qdrant	Filtered hybrid search over 3.2M vectors
Ranking & gating	Retrieval API	Cosine ordering, relevance floor, duplicate collapse
Precision reranking	bge-reranker-v2-m3	Cross-encoder reordering of the candidate set
Answer synthesis	Researcher agent LLM	Composes the response strictly from returned passages

The interface between the platform and the agent is deliberately narrow: a question and an optional scope in, ranked passages with provenance out. The retrieval layer never generates prose, and the agent never queries the vector database directly. Each side can evolve independently.

Critically, the platform can return **nothing**. When no passage clears the relevance floor, the API responds with an empty result set, signalling unambiguously that the corpus does not support an answer — the foundation of a system that declines to speculate.

11. Data Update Pipeline

The knowledge base tracks the authorities continuously. Apache Airflow orchestrates acquisition; the index updates itself from what acquisition produces.



Change detection

The pipeline is incremental by design. Every indexed chunk records the **ETag** of the S3 object it came from — a content fingerprint supplied free with the object listing. Before any work is scheduled, the pipeline compares current ETags against those already indexed and processes only what has genuinely changed.

Deciding to skip a document therefore costs nothing: no download, no parsing, no embedding. A routine cycle over a corpus with a handful of updated documents touches only those documents, and the platform never performs a full re-index.

Update semantics

Event	Behaviour
New document	Extracted, chunked, embedded, inserted
Revised document	ETag differs; chunks regenerated and upserted in place under their deterministic keys
Unchanged document	Skipped before any processing
Withdrawn document	Chunks pruned so retrieval cannot cite material no longer published

Because chunk keys are deterministic, updates overwrite rather than accumulate — the index cannot drift into duplicate states, and the pipeline is safe to re-run at any point. Retrieval continues uninterrupted throughout; indexing and serving share the database but never block one another.

12. Technology Stack

Layer	Technology	Role
Orchestration	Apache Airflow	Scheduled scraping, Dataset-triggered indexing, dependency management
Storage	Amazon S3	Immutable document lake with atomic commit and versioned pointers
Vector database	Qdrant	Hybrid vector search, HNSW indexing, payload filtering
Embedding	BAAI/bge-m3	Dense (1,024-d) and sparse representations
Reranking	BAAI/bge-reranker-v2-m3	Cross-encoder precision stage
Generation	Researcher agent LLM	Grounded answer synthesis from retrieved passages
GPU compute	RunPod (RTX 4090)	On-demand acceleration for bulk embedding
Document processing	PyMuPDF	Text and layout extraction from regulatory PDFs
Service layer	FastAPI · Uvicorn	Retrieval API and operational endpoints
Runtime	Python 3.11 · Docker · systemd	Containerised services under supervised process management

13. System Strengths

Capability	Realisation
Scalable architecture	3.2M vectors served from commodity hardware; quantized indexing keeps memory bounded as the corpus grows
Automated ingestion	Scheduled scrapers, atomic commits and Dataset-triggered indexing — no manual step between publication and searchability
Semantic search	Questions match meaning, not keywords: "safe while breastfeeding" finds "Fertility, pregnancy and lactation"
High retrieval accuracy	Hybrid dense + sparse recall, structure-aware chunking, metadata scoping and cross-encoder reranking compounding at each stage
Multilingual capability	A single shared embedding space across 100+ languages, enabling regional filings without a parallel pipeline
Efficient vector search	Median query latency ~300 ms end to end, including query embedding
Full traceability	Every passage carries source, document, page and regulatory section — no unsourced claim can reach the user
Modular design	Ingestion, indexing, retrieval and generation are independently deployable and independently replaceable
Easy extensibility	A new source is a manifest and a scraper; the indexing path requires no change
Production-ready	Supervised services, atomic commits, idempotent writes, resumable jobs and health endpoints throughout
Continuous knowledge updates	ETag-driven incremental sync propagates newly published material without re-indexing the corpus